artificial-word · v1.0 fundacional · seed:42

MOTOR DE CIVILIZACIONES EMERGENTES

Artificial World

Agentes autónomos por **función de utilidad**. Refugios con **memoria persistente**. Héroes que dejan **crónica verificable**. Sin LLMs en el núcleo.

13	9	200	68+
TIPOS DE ACCIÓN	MODIFICADORES	TICKS CANÓNICOS	TESTS PASANDO

Cosigein SL

2026-03-08 · artificial-word · Semilla 42 · 8 agentes · 60×60

VEREDICTO SISTEMA

TENSIÓN

ÍNDICE DE CONTENIDOS

Lo que encontrarás aquí

FUNDAMENTOS

01 **Antes de empezar** — lo que existe y lo que no 3

02 **El problema** — por qué otro motor de vida artificial 4

03 **Arquitectura** — tres capas, un modelo, sin integración forzada 5

EL MOTOR

04 **Cómo decide un agente** — función de utilidad completa 7

05 **Sesión canónica** — lo que pasó en 200 ticks 10

06 **Tres cosas que nadie programó** — comportamiento emergente 12

INFRAESTRUCTURA

07 **Herramientas invisibles** — Modo Sombra, Watchdog, Chess 14

08 **Las 7 civilizaciones** — semillas arquetipos 16

POSICIÓN

09 **Lo que no hace todavía** — limitaciones documentadas 17

10 **La convicción** — trazabilidad radical como diseño 18

11 **Para reproducir esto** — comandos verificados 19

Ref Referencias 20

CAPÍTULO 01

Antes de empezar: lo que existe y lo que no

La regla que hace que este documento valga algo.

Este documento tiene una regla. Cada vez que se diga que algo "existe" o "funciona", existe y funciona de verdad, verificable con el código. Cuando algo es visión futura, se dice explícitamente.

ESTADO	DEFINICIÓN	EJEMPLOS EN EL PROYECTO
REAL	Testeado, reproducible, en producción	Motor Python · 68+ tests · persistencia SQLite · Modo Sombra
DEMO	Existe y funciona, no es el motor principal	Capa web fullstack · DobackSoft integrado
PARCIAL	Existe con funcionalidad incompleta	HeroRefuge · CivilizationSeed web · IA local (Ollama)
ROADMAP	Diseñado, no implementado	Runtime 3D · integración Python/JS · diplomacia entre civs

REPRODUCIBILIDAD GARANTIZADA

Todo lo que afirma este documento puede verificarse con un solo comando. La misma semilla produce exactamente los mismos resultados en cualquier máquina con Python 3.11+.

BASH — REPRODUCIR LA SESIÓN CANÓNICA

```bash
python cronica_fundacional.py --seed 42 --ticks 200 \
  --founder "Tryndamere" --refuge "Refugio Fundador"
# salida: JSON + Markdown + PDF — idéntico en cada ejecución
```

CAPÍTULO 02

El problema que resuelve

Por qué otro motor de vida artificial.

"El agente no recuerda. No tiene razones. Emerge, pero no decide."

MOTIVACIÓN CENTRAL — SOBRE LOS SISTEMAS CLÁSICOS DE ALIFE

Los sistemas de vida artificial clásicos — el Game of Life de Conway (1970), los Boids de Reynolds (1987) — son elegantes. Priorizan la simplicidad de las reglas sobre la riqueza del estado interno. En los sistemas más complejos — Dwarf Fortress, RimWorld — el mecanismo de decisión tiende a ocultarse bajo capas de abstracción opacas.

Artificial World busca un punto diferente: estado interno rico y decisión completamente transparente.

Función de utilidad pública

Cada decisión de cada agente en cada tick es pública, parametrizada y reproducible. No hay magia invisible.

Memoria persistente entre ticks

Posiciones de recursos y relaciones sociales. Las decisiones pasadas condicionan las presentes.

Ruido determinista por semilla

`random.Random(id × 1000 + tick)`. La misma semilla produce exactamente las mismas decisiones.

Refugio como entidad de primer orden

No es un artefacto del mapa. Tiene nombre, recursos, moral, memoria y fase de crecimiento propia.

CAPÍTULO 03

La arquitectura en una página

Tres capas. Un modelo conceptual compartido. Sin integración forzada.

MODELO CONCEPTUAL COMPARTIDO

World · CivilizationSeed · Refuge · Hero · Community · MemoryEntry · HistoricalRecord · Territory

MOTOR PYTHON — REAL

principal.py · nucleo/simulacion.py · 13 acciones · 68+ tests · SQLite · Modo Sombra · Watchdog · pygame

CAPA WEB — DEMO

Node + Express · React + Vite · HeroRefuge · 7 semillas · IA local (Ollama)

SISTEMA CHESS — AUDITORÍA INDEPENDIENTE

6 agentes Docker · red aislada · montaje :ro · sin estado entre ejecuciones · REPORTE_CHESS_1.md

DECISIÓN ARQUITECTÓNICA EXPLÍCITA

La separación Python/Web no es trabajo pendiente. Está documentada en `docs/DECISION_PUENTE_PYTHON_JS.md`. El coste de implementar el puente en tiempo real supera el beneficio en esta fase. Dos motores, dos propósitos — más honesto que una integración superficial.

Los 12 subsistemas del motor Python

SUBSISTEMA	CLASE	FUNCIÓN
Ticks	`GestorTicks`	Control de tiempo y velocidad de simulación
Eventos	`BusEventos`	Publicación / suscripción entre subsistemas
Métricas	`SistemaMetricas`	Conteo de acciones y estados por tick
Regeneración	`SistemaRegeneracion`	Reposición de recursos en el mapa grid
Persistencia	`SistemaPersistencia`	Guardar / cargar estado en SQLite
Watchdog	`SistemaWatchdog`	Alertas estructuradas de condiciones anómalas
Reporte	`SistemaReporte`	Generación de crónica fundacional (JSON/MD/PDF)
Modo Sombra	`GestorModoSombra`	Simulación especular sin efectos secundarios
Competencia	`SistemaModoCompetencia`	Modo multi-jugador por turnos
Renderizador	`Renderizador`	Representación visual pygame

CAPÍTULO 04

Cómo decide un agente

La función de utilidad completa, sin omisiones.

El ciclo por tick, por entidad

PYTHON — CICLO DE DECISIÓN

```
actualizar_estado_interno()   # hambre++, energía--
percibir_entorno()            # radio local + amenaza_local
actualizar_memoria()          # registra recursos y refugios visibles
actualizar_directivas()       # expira instrucciones caducadas
ContextoDecision(tick, mapa)  # empaqueta todo
decidir_accion()              # U(a,i,t) → acción ganadora
ejecutar_accion()             # aplica efectos en el mundo
```

LA FÓRMULA CENTRAL

$$U(a, i, t) = U_{base}(a) + \Sigma_{k=1..9}\, \delta_k(a,i,t) + \varepsilon(i,t)$$

Donde $\varepsilon(i,t)$ = `random.Random(id × 1000 + t).uniform(-0.08, 0.08)` — ruido determinista por semilla.

Las 13 acciones y sus utilidades base

ACCIÓN	U_{BASE}	CONDICIÓN DE VIABILIDAD
COMER	0.65	Comida en inventario > 0
RECOGER_COMIDA	0.55	Celda actual tiene recurso COMIDA
EXPLORAR	0.50	Energía > 0.20
HUIR	0.40	Riesgo percibido > 0.3
DESCANSAR	0.30	*Fallback garantizado — siempre viable*
MOVER	0.30	Posición vecina disponible
RECOGER_MATERIAL	0.30	Celda actual tiene recurso MATERIAL
IR_REFUGIO	0.25	Refugio adyacente + (energía < 0.45 OR riesgo > 0.5)
EVITAR	0.25	Riesgo percibido > 0.3
COMPARTIR	0.20	Inventario ≥ 3 OR (cooperativo + inv ≥ 1 + hambre < 0.5)
ROBAR	0.20	Oportunista/agresivo + hambre ≥ 0.75 + sin comida
SEGUIR	0.20	Cooperativo/neutral + entidades cercanas visibles
ATACAR	0.18	Condición contextual agresiva

Los 9 modificadores contextuales

#	MODIFICADOR	VARIABLE FUENTE	EFECTO EN U(A,I,T)
1	mod_hambre	estado.hambre	Escala COMER/RECOGER según hambre actual
2	mod_energia	estado.energia	Aumenta DESCANSAR cuando energía < 0.6
3	mod_riesgo	riesgo_percibido	Activa HUIR/EVITAR cuando riesgo > 0.3
4	mod_rasgo	rasgo_principal	Ajusta preferencias por arquetipo conductual
5	mod_anti_oscilacion	posicion_anterior	Penaliza −0.40 retornar a la celda anterior
6	mod_memoria	memoria.recursos	Bonus +0.15–0.25 hacia recursos recordados
7	mod_relaciones	relaciones[i,j]	Confianza → SEGUIR; hostilidad → HUIR/EVITAR
8	mod_directivas	directivas_activas	Instrucciones externas del investigador
9	mod_autonomia	múltiple	**Override de emergencia — supervivencia anula directivas**

EL MODIFICADOR 9 ES EL MÁS IMPORTANTE DEL SISTEMA

Si hambre > 0.85 → penaliza EXPLORAR aunque haya directiva contraria.
Si energía < 0.15 → penaliza MOVER y ROBAR.
Si riesgo > 0.80 → bonifica HUIR sobre cualquier orden recibida.
Ningún agente puede ser enviado a su muerte por una directiva mal calibrada.

Los 5 rasgos de los agentes sociales

RASGO	POTENCIA	PENALIZA
COOPERATIVO	COMPARTIR +0.25 · SEGUIR levemente	ROBAR −0.25
NEUTRAL	*Sin modificaciones*	
AGRESIVO	ROBAR +0.30	COMPARTIR −0.20 · HUIR −0.10
EXPLORADOR	MOVER +0.35 · EXPLORAR +0.35	SEGUIR levemente
OPORTUNISTA	ROBAR +0.15 · RECOGER_MATERIAL +0.08	—

CAPÍTULO 05

Lo que pasó en la sesión canónica

Semilla 42 · Mapa 60×60 · 7 sociales + 1 gato · 200 ticks.

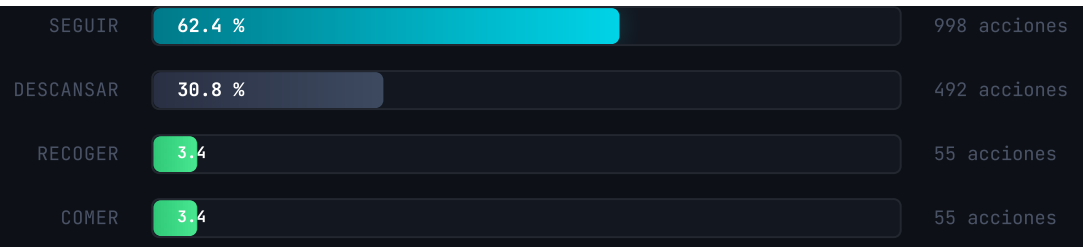
8/8	0.95	65	TENSIÓN
AGENTES VIVOS	HAMBRE PROMEDIO	ALERTAS WATCHDOG	VEREDICTO

Estado final de la población (tick 200)

AGENTE	TIPO	POSICIÓN	HAMBRE	ENERGÍA	ESTADO
Ana	social	(47, 21)	0.68	0.38	Advertencia
Bruno	social	(0, 34)	1.00	0.36	Crítico
Clara	social	(25, 5)	1.00	0.26	Crítico
David	social	(10, 22)	0.90	0.18	Crítico
Eva	social	(13, 5)	1.00	0.40	Crítico
Félix	social	(0, 52)	1.00	0.38	Crítico
Amiguisimo	gato	(12, 42)	1.00	0.30	Crítico
Tryndamere	social	(2, 51)	1.00	0.40	Crítico

Salud = 1.0 para todos. Ningún agente alcanzó el estado de muerte. Hambre: 0 = saciado · 1 = crítico.

Distribución de 1.600 acciones



EL HALLAZGO PRINCIPAL

Bajo hambre crítica generalizada, el **62.4% de las acciones fue SEGUIR** — seguirse los unos a los otros. No fue codificado como respuesta a la escasez. *Emergió.*

CAPÍTULO 06

Tres cosas que nadie programó

Comportamiento emergente verificable.

1 · La trampa dinámica de la cohesión

SEGUIR dominando al 62.4% bajo escasez severa no fue diseñado como respuesta a la escasez. Es el resultado de tres mecanismos interactuando:

- **mod_relaciones:** Los agentes con historia de cooperación tienen confianza alta. Confianza alta → bonus SEGUIR. La señal social supera la señal de hambre.
- **mod_autonomia:** Con hambre > 0.85, el override penaliza EXPLORAR. El agente hambriento no puede explorar agresivamente aunque quiera.
- **Ausencia de recursos:** COMER y RECOGER no son candidatos la mayor parte del tiempo. Las celdas están vacías.

LA TRAMPA

Los agentes cohesionados se mueven juntos hacia zonas que el propio grupo ha agotado. La confianza social supera la señal de hambre. Análogo al *information cascade* de Bikhchandani et al. (1992): se imita al grupo incluso cuando esa imitación es colectivamente subóptima.

2 · El agotamiento zonal del cuadrante oeste

Las 65 alertas se concentran en x < 15, y > 40: Tryndamere (2,51), Félix (0,52), Amiguisimo (12,42). La semilla 42 genera un gradiente de recursos que desfavorece el borde oeste.

EFECTO LATERAL NO ESPERADO

`mod_anti_oscilacion` penaliza −0.40 regresar a la celda anterior. Diseñado para evitar el "ping-pong". Funciona. Pero en agotamiento zonal total, ese mismo modificador dificulta el retroceso hacia zonas con recursos. **Una regla que mejora el comportamiento promedio crea un caso extremo patológico.**

3 · Ana y la divergencia individual exitosa

Ana termina con hambre **0.68** — la única sub-crítica. Posición: (47, 21), ~38 unidades del centroide del grupo. Nadie la envió ahí. No hay directiva. Es el resultado acumulado de exploración individual que la llevó al cuadrante noreste, donde el grupo nunca llegó.

> IMPLICACIÓN TEÓRICA
>
> La divergencia individual de la cohesión grupal puede ser una estrategia emergentemente superior bajo presión de escasez, aunque el sistema no la codifica como tal. Un agente EXPLORADOR reproduce el patrón de Ana por diseño. Un COOPERATIVO tiende a quedar atrapado en la trampa.

Tabla de fenómenos emergentes

FENÓMENO	MECANISMO GENERADOR	¿PROGRAMADO?
Cohesión bajo escasez	mod_relaciones > mod_autonomia en estadios medios	NO
Agotamiento zonal	mod_anti_oscilacion + gradiente de recursos	PARCIAL
Divergencia exitosa	rasgo_explorador + exploración no competida	NO
Supervivencia total	mod_autonomia activa en estados críticos	SÍ
Fragmentación en 3 clusters	Dispersión exploratoria + gradiente de recursos	PARCIAL

CAPÍTULO 07

Las herramientas que nadie ve

Modo Sombra · Watchdog · Sistema Chess.

Modo Sombra

Simulación paralela que replica el estado del mundo principal y acepta intervenciones sin modificar lo observable. Permite evaluar contrafácticos: *¿qué habría pasado si David hubiera tomado la decisión Y en el tick 147?*

- **Contrafácticos:** evaluación sin contaminar la simulación principal
- **Control manual WASD:** el investigador toma el control de un agente; la IA cede el turno
- **Modo investigador:** directivas experimentales en entorno aislado

Watchdog

Cada alerta tiene estructura formal verificable contra el log de acciones del mismo período:

JSON — ALERTA WATCHDOG REAL (SESIÓN CANÓNICA, TICK 189)

```
{
  "nivel":          "CRITICAL",
  "codigo":         "HAMBRE_CRITICA_SIN_RESPUESTA",
  "tick":           189,
  "entidad_nombre": "Bruno",
  "descripcion":    "Hambre ≥ 0.9 durante 8 ticks sin COMER"
}
```

Sistema Chess — auditoría independiente

El mismo equipo que construye no puede auditarse a sí mismo con objetividad. El Sistema Chess resuelve esto con 6 agentes dockerizados en red aislada, sin acceso de escritura.

AGENTE	ESPECIALIDAD	DETECTA
agent-docs	Documentación	Claims sin evidencia · enlaces rotos
agent-backend	API / seguridad	`console.log` · URLs hardcodeadas · catch vacíos
agent-frontend	React	Componentes >300 líneas · imports incorrectos
agent-bd	Base de datos	SQL injection · SELECT * · sin IF NOT EXISTS
agent-tests	Cobertura	Tests sin assertions · ratio <30%
agent-marketing	Narrativa	Claims sin evidencia · overselling

AISLAMIENTO DE RED

prod-net: backend + frontend (3001/5173) · **tests-net:** python-tests + backend-tests (3002) · **audit-net:** 6 agentes + coordinator, sin acceso a prod ni a tests. Repositorio montado en `:ro`.

CAPÍTULO 08

Las 7 civilizaciones posibles

Semillas arquetipos que condicionan la historia antes del tick 0.

Cada semilla no es solo un nombre. Condiciona valores, tensiones internas, el tipo de héroe probable y el tono de la historia que emerge.

ARQUETIPO	VALORES CENTRALES	TENSIÓN PRINCIPAL	HÉROE PROBABLE
🏕️ **Tribu Fronteriza**	Adaptación · cooperación	Clima hostil · escasez	Explorador / Guardián
🔬 **Refugio Tecnócrata**	Orden · eficiencia · conocimiento	Rigidez · elitismo	Ingeniero / Estratega
🛕 **Comunidad Espiritual**	Fe · vínculo · memoria	Dogma · miedo al cambio	Guardián / Profeta
⚔️ **Reino Guerrero**	Honor · fuerza · jerarquía	Expansión · coste humano	Campeón / Conquistador
🏛️ **Ciudad Mercante**	Intercambio · movilidad	Desigualdad · corrupción	Negociador / Explorador
🔒 **Colonia Paranoica**	Seguridad · vigilancia	Sospecha · fractura interna	Vigía / Ingeniero
👑 **Imperio Decadente**	Linaje · poder · apariencia	Declive · intriga	Heredero / Diplomático

EL FLUJO FUNDADOR MÍNIMO

Elige semilla → nombra héroe → nombra refugio. El sistema crea: Hero · Refuge (food=60, shelter=75, security=55, morale=58) · Community · MemoryEntry inicial · World con tick=0. Tiempo total: menos de 2 minutos desde el onboarding.

CAPÍTULO 09

Lo que el sistema no hace todavía

Limitaciones documentadas, sin rodeos.

- **Los dos motores no están integrados.** Python y la capa web comparten modelo conceptual pero no hablan en tiempo real. Es una decisión documentada, no un olvido. ROADMAP
- **Community existe como contrato de datos.** Los grupos tienen tensiones declaradas pero no simuladas tick a tick. PARCIAL
- **No hay diplomacia entre civilizaciones.** Guerras, linajes, migraciones entre mundos. ROADMAP
- **El runtime 3D no existe.** La verdad sistémica vive en 2D. La encarnación visual con héroes y eventos históricos visuales es la capa futura. ROADMAP
- **Las crónicas no incluyen capturas automáticas de ticks clave.** La historia se genera en texto y JSON, pero sin snapshots visuales de los momentos críticos. ROADMAP

CAPÍTULO 10

La convicción detrás del diseño

Por qué la trazabilidad radical es una propiedad de primer orden.

"Nunca ha sido más fácil generar texto convincente sobre sistemas. Y nunca ha sido más difícil distinguir qué parte de un sistema existe de verdad de qué parte existe solo como afirmación."

MANIFIESTO FUNDACIONAL · ARTIFICIAL WORLD

Artificial World parte de la premisa contraria: **la trazabilidad radical es una propiedad de diseño de primer orden, no una característica opcional.**

🔗

Cada comportamiento es reproducible

Mediante semilla fija. No hay decisiones que no puedan reconstruirse desde el estado inicial.

📌

Cada claim apunta a código

Si se dice que hay 13 acciones, hay 13 clases en `acciones/`. Si el Watchdog emite alertas estructuradas, hay tests que lo verifican.

🗂️

Taxonomía real / demo / roadmap

Pública, verificable por cualquier persona que lea el código. No es una concesión — es una herramienta activa de gestión de producto.

🎭

Chess como auditoría institucional

6 agentes que no pueden ser influenciados por el equipo que construye. El reporte es verificable, reproducible y fechado.

MANIFIESTO DEL PROYECTO

"Una respuesta sin rastro no es una base seria de decisión."

CAPÍTULO 11

Para reproducir exactamente esto

Comandos verificados. Sin sorpresas.

POWERSHELL — SESIÓN CANÓNICA (MOTOR PYTHON)

```
python cronica_fundacional.py --seed 42 --ticks 200 \
  --founder "Tryndamere" --refuge "Refugio Fundador"
# Duración: < 30 segundos · Salida: JSON + Markdown + PDF
```

POWERSHELL — TESTS COMPLETOS (68+ TESTS, 11 SUITES)

```
$env:SDL_VIDEODRIVER = "dummy"
$env:SDL_AUDIODRIVER = "dummy"
python pruebas/run_tests_produccion.py
```

POWERSHELL — FULLSTACK WEB

```
.\scripts\iniciar_fullstack.ps1
# Backend: puerto 3001 · Frontend: puerto 5173
# Demo pública: https://artificialworld.es
```

REFERENCIAS

Trabajo relacionado

Las bases sobre las que se construye este sistema.

[1] Adams, T. (2006). *Dwarf Fortress.* Bay 12 Games.

[2] Bikhchandani, S., Hirshleifer, D., & Welch, I. (1992). A Theory of Fads, Fashion, Custom, and Cultural Change as Informational Cascades. *Journal of Political Economy,* 100(5), 992–1026.

[3] Conway, J. H. (1970). The Game of Life. *Scientific American,* 223(4), 4–10.

[4] Langton, C. G. (1989). Artificial Life. *SFI Studies in the Sciences of Complexity,* 1–47.

[5] Rao, A. S., & Georgeff, M. P. (1991). Modeling Rational Agents within a BDI-Architecture. *KR '91,* 473–484.

[6] Reynolds, C. W. (1987). Flocks, Herds, and Schools: A Distributed Behavioral Model. *SIGGRAPH,* 21(4), 25–34.

[7] Sylvester, T. (2013). *RimWorld.* Ludeon Studios.

[8] Von Neumann, J., & Morgenstern, O. (1944). *Theory of Games and Economic Behavior.* Princeton University Press.

Artificial World · Motor Python + React · Agentes autónomos por utilidad · Sin LLMs en el núcleo

artificial-word · v1.0 · 2026-03-08